Uncertainty Approximation in Recommendation System

(Dedicated for the Honors degree in Joint Math and Computer Science)

Charlie Chen[1*]

[1] College of Art and Science (CAS), New York University

Abstract

Abstract In this study, we investigate the effect of Bayesian Machine Learning on the recommendation task. Our approach is inspired by the work of [1], in which the authors propose the use of dropout to efficiently compute approximations for both the mean and variance of the predictive distribution. We extend the results from [1] and show that they are valid in the context of recommendation tasks, up to some approximations. To further explore the utility of our approach, we conduct an extensive empirical experiment on a small, dense dataset. In this experiment, we vary the dropout ratio and number of forward passes, and observe the resulting relationship between the predictive variance, dropout ratio, and number of forward passes. Our results provide valuable insights into the effectiveness of using Bayesian Machine Learning and dropout for recommendation tasks.

Keywords: *Bayesian Machine Learning; Recommendation System; MC Dropout*

1. Introduction

Since deep learning shows its unexpected well performance in 2012 [2], it has led to many breakthroughs in various fields, including image and speech recognition, natural language processing. A particular interesting application of deep learning is in recommendation system, which is a tool that uses data about user preferences, behavior, and interactions to suggest items that the user may like. These systems are commonly used in a variety of applications, such as online shopping, music and video streaming, and social media. The upgrade of hardware and internet infrastructures produce a lot of multimodal data (clicks, texts, images) and enable large scale training of deep learning models. Though recommendation systems benefit from deep learning and have remarkable results, it suffers from cold-start and sparsity problem. For the sparsity problem, consider a matrix, where each row represents a user and each column represent an item. Each entry represents how many times the user has clicked the items. Note that this matrix may be enormous (consider the number of items and users on Amazon) and most entries are zero. For the zero entries, the user is either disinterested in the product or interested in the product but never exposed. So the model (or actually humans) cannot know the exact reasons for the zero entries. The only way to test this is A/B testing, which is expensive and inefficient.

[*]**Corresponding author.** *E-mail address: zc2157@nyu.edu*

Bayesian methods provide a potential solution to this dilemma. In Bayesian machine learning, the prediction of uncertainty is automatic and built-in. Therefore, with the utilization with Bayesian methods, we hope to compute the uncertainty for recommendations. Nonetheless, if we consider the model family to be neural networks, since the scale of parameters is large, exact Bayesian inference is usually intractable. Approximations are usually required in practice. In this paper, we attempt to approximate the inference results by adding drop out to the model [1]. To the best knowledge of the authors, this is the first attempt to utilize Bayesian machine learning to estimate the uncertainty in recommendation.

2. Preliminary

2.1. Recommendation System

The problem of recommendation system is usually presented as a matrix factorization and entry filling problem [3, 4]. The gist behinds the idea is that there is a hidden representation for the users and the items so that their inner product can represent the probability of user's preference. This kind of methods is called Collaborative Filtering (CF), or rating-based methods. With the advent of deep learning, a series of works have shown great potentials of deep learning methods[5, 6, 7]. Deep learning shows its effectiveness because it provides a good representation for users and items and non-linear interaction function that replaces the previous inner product.

Though these deep learning techniques demonstrate their great potentials, the inherent issue in the problem of recommendation is the data sparsity and cold-start problem. When the interaction data is not been collected or the item is newly published, we have no available data to do collaborative filtering, so in this case, we need to utilize the content of the item to make recommendations [8]. This direction of recommendation is called content-based methods. The combination of rating-based and content-based methods is called hybrid model, which is a particularly interesting topic for research in the fields of deep recommendation system because it both utilizes users' feedback and the intrinsic content for users and items, which make the recommendation system more robust and ameliorate the cold-start problem.

A general rule of thumb of combining the content information and the rating information is using one random variable to compute different objectives: one for content prediction and one for rating prediction. For scientific articles, Collaborative topic regression (CTR) models add the topic vector to the item embedding and use the topic vector to compute the word assignment [9]. Later works adapt the idea of denoising. In particular, [10] feeds corrupted input to the encoder, uses the encoded vector for rating prediction as well as input reconstruction: again, one loss for rating and one loss for content. [11] uses similar idea but they utilizes variational autoencoder to add stochasticity to the latent vector.

In summary, the framework for recommendation is as follow. Shared low dimensional space for latent vector is K. The latent vector for user i is defined as

$$u_i \sim \mathcal{N}\left(0, \lambda_u^{-1} I_K\right) \tag{1}$$

The latent vector for item is defined as

$$v_j^{\dagger} \sim \mathcal{N}\left(0, \lambda_v^{-1} I_K\right) \tag{2}$$

Additionally, there may be a content vector defined as

$$z_j \sim \mathcal{N}\left(0, I_K\right) \tag{3}$$

The content latent vector is used to predict the content of the item (usually reconstruct).

$$x_j \sim p_\theta\left(x_j \mid z_j\right) \tag{4}$$

Similarly, given the content of the item, we can predict the latent vector

$$z_j \sim p_\theta \left(z_j \mid x_j \right) \tag{5}$$

The content latent vector is added to the vector latent vector to represent the item vector.

$$v_j = v_j^\dagger + z_j \tag{6}$$

Finally, the rating is drawn from the normal distribution.

$$R_{ij} \sim \mathcal{N} \left(u_i^T v_j, C_{ij}^{-1} \right) \tag{7}$$

where C_{ij} represents the precision and is usually values like number of clicks.

The loss, or the target, of the task is in the form of

$$L = L_{\text{rating fitting}} + L_{\text{content fitting}} + L_{\text{regularization}} \tag{8}$$

Recently, a novel approach called DropoutNet has been proposed to address the cold start problem by applying dropout regularization to the input before it is fed into the neural network encoder [12]. Despite its simplicity, this method has been shown to provide effective performance in both cold start and warm start cases. By randomly dropping out existing content information, DropoutNet simulates the sparsity of content information and improves the performance of the model. An added advantage of this technique is that it does not have any requirements or constraints on the model architecture, allowing it to be applied to a wide range of models with only simple adaptations.

2.2. Bayesian Machine Learning

Bayesian machine learning is a probabilistic approach to analyzing and learning from data. It is based on the Bayesian theorem, which provides a mathematical framework for updating our beliefs about the state of the world based on new evidence. In Bayesian machine learning, probability distributions are used to encode our prior beliefs about the model parameters, and the model uses observed data to update these distributions and arrive at a posterior distribution that reflects our updated beliefs. This posterior distribution is used to make predictions about future data.

One key advantage of Bayesian machine learning is that it allows us to quantify our uncertainty about the model parameters, which can be useful for decision-making. For example, in a medical setting, a Bayesian model could be used to predict the probability that a patient has a certain disease, taking into account not only the observed symptoms, but also our prior knowledge about the prevalence of the disease in the population.

Bayesian machine learning also allows us to naturally incorporate prior knowledge into our models. This can be especially useful when we have limited data, as we can leverage our existing knowledge to make more accurate predictions. Additionally, Bayesian models are often more robust to overfitting than other machine learning methods, as they can incorporate regularization through the use of prior distributions.

Then, we formally represent the framework of Bayesian machine learning. Assume the prior for the parameter is

$$p(\mathbf{w}) \sim \mathcal{N} \left(0, \gamma^{-1} I \right) \tag{9}$$

The likelihood for the data given the model and weight is

$$p(\mathbf{y} | \mathbf{X}, \mathbf{w}) \sim \mathcal{N} \left(\mathbf{y}; \hat{\mathbf{y}}(\mathbf{X}, \mathbf{w}), \tau^{-1} I \right) \tag{10}$$

So, by Bayesian theorem, the posterior can be obtained by

$$p(\mathbf{w} \mid \mathbf{y}, \mathbf{X}) = \frac{p(\mathbf{y} \mid \mathbf{X}, \mathbf{w})\, p(\mathbf{w})}{p(\mathbf{y} \mid X)} \tag{11}$$

If $\hat{\mathbf{y}}(\mathbf{X}, \mathbf{w})$ is linear with respect to $\mathbf{w}$, then the posterior is also Gaussian. In this case, we call that the likelihood and the prior is likelihood, so we can compute the posterior analytically. However, this is not always the case, especially since $\hat{\mathbf{y}}$ is frequently represented as a neural network, which is highly nonlinear. Therefore, in these situations, we need approximation methods like Markov chain Monte Carlo (MCMC) or variational inference.

2.3. Dropout as a Bayesian Approximation

MCMC and variational inference are two commonly used methods for approximate Bayesian inference. But both methods have some drawbacks.

One drawback of MCMC methods is that they can be slow to converge and expensive to compute, especially for complex models with many parameters. This can make them impractical for real-time applications or for very large datasets. Additionally, MCMC methods can sometimes exhibit poor mixing, which means that they may not explore the entire posterior distribution and can produce biased estimates of the model parameters.

Variational inference methods, on the other hand, can be faster than MCMC methods, but they often require careful choice of the variational distribution and can be sensitive to the initial values of the variational parameters. Additionally, variational methods can sometimes produce approximate posteriors that are overly simplified, which can lead to biased estimates of the model parameters.

One influential method that comes up lately is MC Dropout [1]. MC Dropout is a method for approximate Bayesian inference in neural networks. It is based on the idea of using dropout regularization as a way to incorporate uncertainty into the predictions made by a neural network.

Dropout is a regularization technique that randomly "drop out" or ignore some coordinates of the intermediate output vector during training. This has the effect of making the network more robust to overfitting and can also improve the generalization performance of the model. In MC Dropout, the dropout regularization is used at test time, where the network is repeatedly evaluated with different dropout masks to generate a distribution of predictions. This distribution is proved to approximate the Bayesian inference under the assumption of large number of hidden units and small variance for the variational family [1]. Then, this distribution of predictions can be used to estimate the uncertainty of the network's predictions.

One advantage of MC Dropout is that it is computationally efficient and easy to implement, as it can be applied to any pre-trained neural network without the need for additional training. This amazing result echos with the DropoutNet, which uses dropout to solve cold-start problem.

3. Methods

The method described here involves using a dropout approximation to estimate the uncertainty of a recommendation. The vector representation of a user and an item are both assumed to be normally distributed, and the posterior distribution of the item latent vector is also assumed to be normally distributed (5). The inner product of two normal distributions is not itself a normal distribution (6), so the assumption is made that either the user or item are fixed and the rating is normally distributed. Dropout can then be used to calculate the uncertainty of the recommendation.

We want to see the relationship between the level of uncertainties and the dropout ratio. Also, including uncertainty estimation in regression tasks can boost the performance [1]. So, we want to see how the dropout ratio and MC Dropout sampling sizes (equivalently, this is called the number of forward passes because each sampling is one run for the model) effect the accuracy.

3.1. Model architecture

We use the classical structure in [5] and try to see the difference in performance before and after adding the Bayesian approximation by dropout.

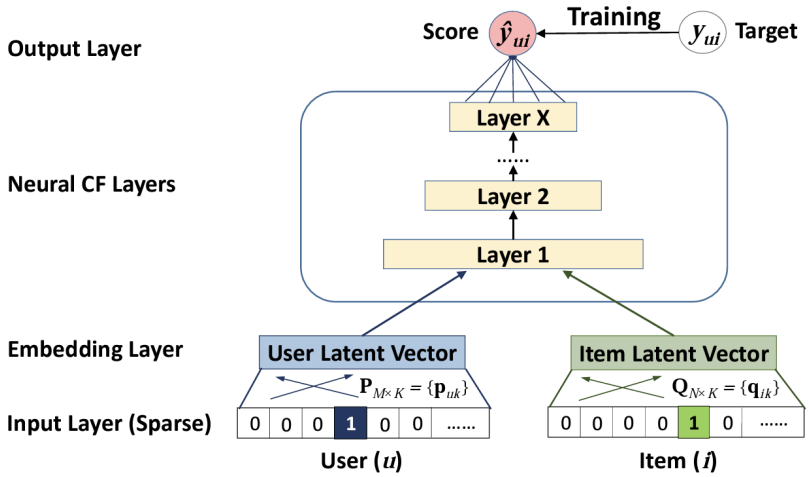

Figure 1. Neural Collaborative Filtering[5]: The user and item are first passed through an embedding layer to project to a high-dimensional space. Then their vector representations are passed into a Multi-layer Perceptron (MLP) to estimate the rating of the user for the item.

4. Experiments

4.1. Dataset

The dataset is collected from a social experiment, in which about 1000 participants are asked to listen 1000 songs and rate them. One good feature about this dataset is that it is dense, so it is useful in areas like causal inference. However, in our situations, this dataset is handful because it presents a regression task for us. We need to make prediction of the rating given the user and the item.

4.2. Metrics

We are using Mean Square Errors (MSE) as our metric since its the typical choice for regression task. From the statistic perspective, minimizing MSE is equivalent to Maximum Likelihood Estimate (MLE).

4.3. Results

4.3.1. Loss Visualization

One thing we particularly care about is the performance of the model, after we add the dropout. In Figure 2, we see that as the dropout ratio increases, the loss increases, meaning that the dropout ratio is negatively related to the loss. This negative impact maybe due to the small size of the dataset. The model is underfitted because the data has a small scale. In Figure 3, the relationship of loss and forward passes is uncertain. Also, notice that the value in y is in a small range. So the variation here is not significant.

4.3.2. Uncertainty Visualization

We are also curious about the relationship between the uncertainty and the dropout ratio and forward passes. In [1], the precision of the dropout model $\tau = pl^2/(2N\lambda)$, where p is the dropout

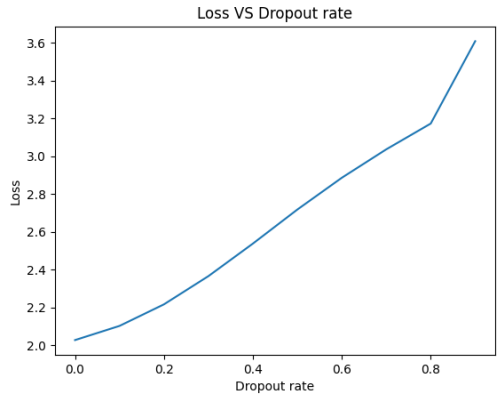

Figure 2. Loss VS Dropout

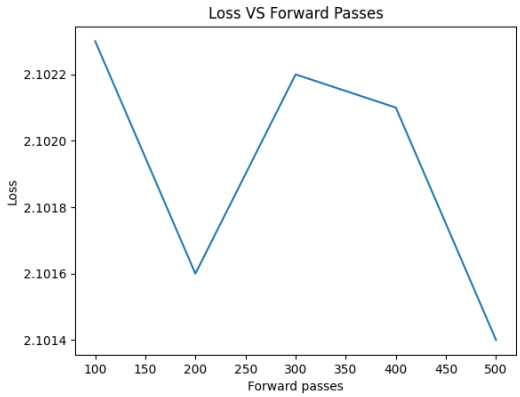

Figure 3. Loss VS Forward Passes

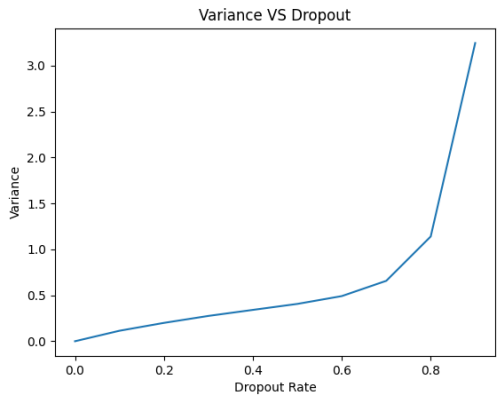

Figure 4. Variance VS Dropout

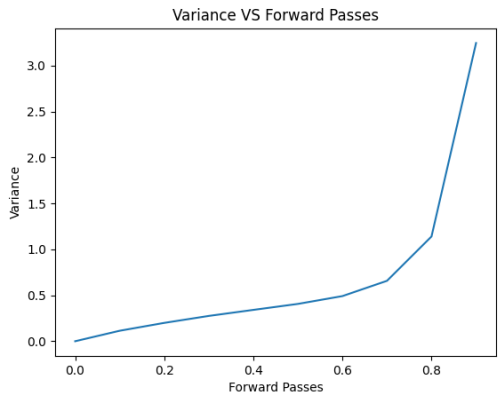

Figure 5. Variance VS Forward Passes

ratio and N is the number of forward passes. When p is increased, the precision of the model increases, so we are getting a more accurate result. This means larger variance because with larger variance, with higher probability, the true value is within posterior distribution. On the other hand, if N is large, the precision is smaller, so the model is less confident, so the variance for predictions is higher. One intuitive way to understand this is that when we want to be confident about something, we usually set a relatively large range so we are probably right.

In Figure 4, we see that when dropout rate increases, the level of uncertainty increases. In Figure 5, we we see that when number of forward passes increases, the level of uncertainty decreases. Both figures fit our expectations.

In Figure 6, we see that the uncertainty blow off at certain points. The effect of dropout ratio is larger than that of number of forward passes.

Next, we want to see if there is a relationship between uncertainty and the true label. Intuitively speaking, the extreme ratings should be more confident while the middle ratings should be less confident (i.e. more uncertainty). In Figure 7, we see that the higher the rating, the larger the uncertainty. One thing worth notice is that there are a lot of outliers in the box plot, which may affect the mean a lot. One thing to notice that, the scatteredness of the variance is smallest for the middle label and larger for the low and high ratings.

Lastly, we want to show the histogram of uncertainty for songs and users. We are curious if any song receive more uncertain ratings than others. In Figure 8 and 9, we see that the predictive variance is approximately normal distributed.

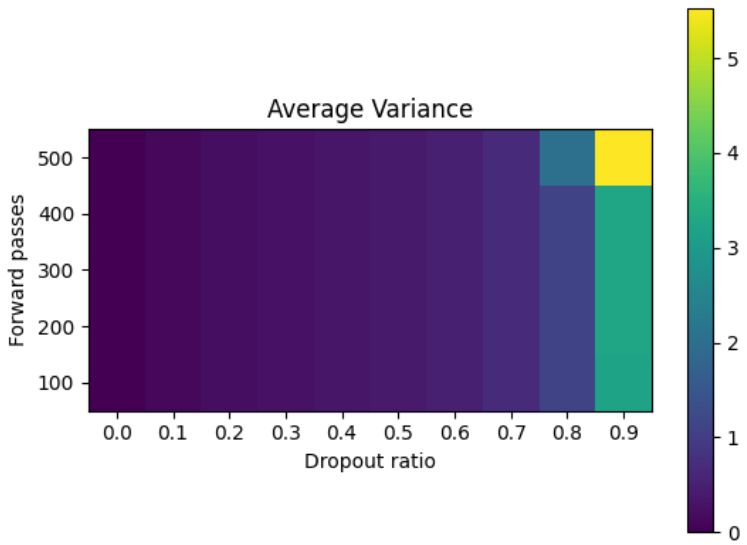

Figure 6. Variance Matrix: the x-axis represents the dropout ratio and the y-axis represents the number of forward passes. The lighter the entry is, the larger the uncertainty for the configuration is.

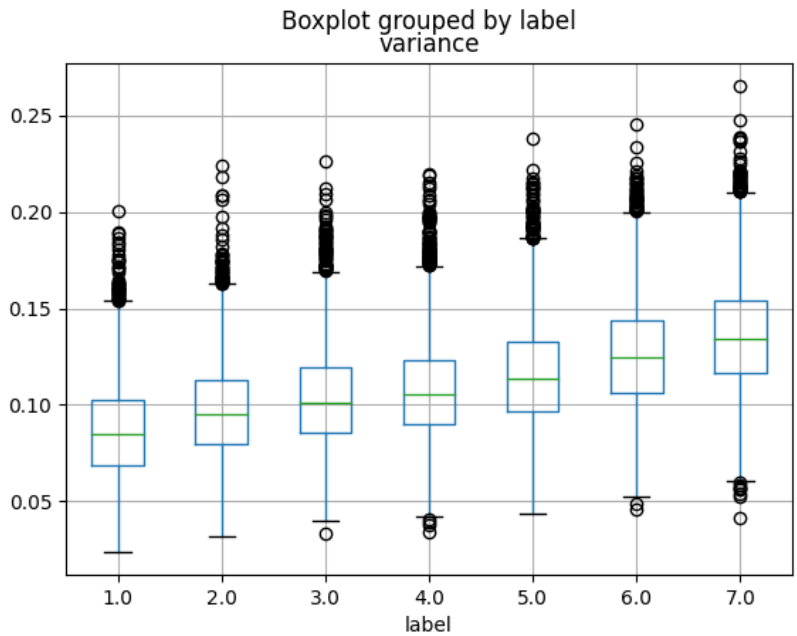

Figure 7. Boxplot for Variance VS Label: the x-axis is the rating of the songs and the y-axis is the variance.

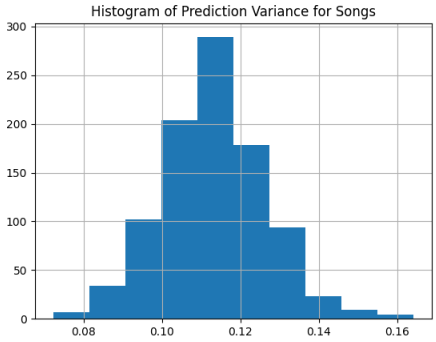

Figure 8. Histogram for Predictive Variance of Songs

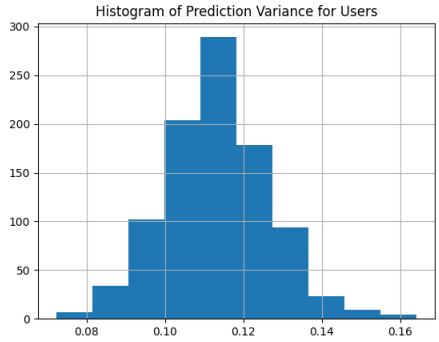

Figure 9. Histogram for Predictive Variance of Users

5. Conclusion and Future Works

In this paper, we examine the effectiveness of Bayesian Machine Learning on a small rating dataset for music. To do so, we use dropout as an approximation for the posterior distribution to obtain both the predictive mean and variance. On the theoretical side, we summarize the work presented in [1] and prove that, in the context of recommendation tasks, we can still approximate the rating as a Gaussian distribution, even though the product of two normal Gaussian variables may not be Gaussian.

We then conduct extensive empirical experiments on our results, varying the dropout ratio and number of forward passes. We find that, in our small dataset, dropout does not improve the validation loss of mean squared error, possibly due to the model being underfitted rather than overfitted. However, we do observe the expected relationship between the predictive variance, dropout ratio, and number of forward passes. We also attempt to identify patterns in the predictive variance distribution and explore ways to utilize uncertainty information.

Moving forward, we plan to conduct our experiments on a larger dataset, such as the Million Songs Dataset, to assess whether dropout is more useful in this scenario due to the abundance of data. We hope to uncover additional patterns in the predictive variance on the larger dataset. On the theoretical side, we aim to refine our approximation of the normal distribution to be more applicable and potentially bound the error from approximation.

References

[1] Y. Gal and Z. Ghahramani, "Dropout as a bayesian approximation: Representing model uncertainty in deep learning," in *international conference on machine learning*. PMLR, 2016, pp. 1050–1059.

[2] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems*, F. Pereira, C. Burges, L. Bottou, and K. Weinberger, Eds., vol. 25. Curran Associates, Inc., 2012. [Online]. Available: https://proceedings.neurips.cc/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf

[3] J. Bennett, S. Lanning *et al.*, "The netflix prize," in *Proceedings of KDD cup and workshop*, vol. 2007. New York, 2007, p. 35.

[4] Y. Koren, R. Bell, and C. Volinsky, "Matrix factorization techniques for recommender systems," *Computer*, vol. 42, no. 8, pp. 30–37, 2009.

[5] X. He, L. Liao, H. Zhang, L. Nie, X. Hu, and T.-S. Chua, "Neural collaborative filtering," in *Proceedings of the 26th international conference on world wide web*, 2017, pp. 173–182.

[6] H.-T. Cheng, L. Koc, J. Harmsen, T. Shaked, T. Chandra, H. Aradhye, G. Anderson, G. Corrado, W. Chai, M. Ispir *et al.*, "Wide & deep learning for recommender systems," in *Proceedings of the 1st workshop on deep learning for recommender systems*, 2016, pp. 7–10.

[7] H. Guo, R. Tang, Y. Ye, Z. Li, and X. He, "Deepfm: a factorization-machine based neural network for ctr prediction," *arXiv preprint arXiv:1703.04247*, 2017.

[8] A. Van den Oord, S. Dieleman, and B. Schrauwen, "Deep content-based music recommendation," *Advances in neural information processing systems*, vol. 26, 2013.

[9] C. Wang and D. M. Blei, "Collaborative topic modeling for recommending scientific articles," in *Proceedings of the 17th ACM SIGKDD international conference on Knowledge discovery and data mining*, 2011, pp. 448–456.

[10] H. Wang, N. Wang, and D.-Y. Yeung, "Collaborative deep learning for recommender systems," in *Proceedings of the 21th ACM SIGKDD international conference on knowledge discovery and data mining*, 2015, pp. 1235–1244.

[11] X. Li and J. She, "Collaborative variational autoencoder for recommender systems," in *Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining*, 2017, pp. 305–314.

[12] M. Volkovs, G. Yu, and T. Poutanen, "Dropoutnet: Addressing cold start in recommender systems," *Advances in neural information processing systems*, vol. 30, 2017.